Analytics SDK & Dashboard — Assignment aur Interview Guide

Yeh document Little Olive restaurant demo ke analytics assignment ko samjhata hai: system ka purpose, React aur Laravel ka role, event ka browser se dashboard tak ka safar, database schema, API contract, local setup, tests, engineering decisions aur interview mein project ko explain karne ke points.

1. Assignment ka goal

Ek chhota end-to-end analytics system banana jo:

1. Browser se page views aur custom user activity capture kare.
2. Server se business events, jaise successful order, record kare.
3. Events ko validate karke database mein persist kare.
4. Dashboard par traffic, unique visitors, popular pages aur event trends dikhaye.
5. Restaurant demo ke zariye user interaction se reporting tak ka flow dikhaye.

2. Project mein istemal technologies

Hissa	Technology	Is project mein kaam
Frontend	React 18	Restaurant UI, order form aur analytics dashboard
Frontend dev/build server	Vite 6	React app serve aur production bundle build karta hai
Backend	Laravel 12 / PHP 8.2+	JSON APIs, validation, business logic aur analytics queries
Database	SQLite	Demo ke orders aur events ko local file mein persist karta hai
PHP dependencies	Composer	Laravel aur uske PHP packages install karta hai
Frontend dependencies	npm	React, Vite aur frontend packages install karta hai

Vite development mode mein `/api` requests ko Laravel ke `127.0.0.1:8000` server tak proxy karta hai. Production mein frontend aur API ek hi host par ya alag hosts par deploy ho sakte hain; alag frontend host ke liye API base URL aur Laravel CORS origins configure karne honge.

3. Architecture: kaunsa hissa kya karta hai?

```
Browser
  React restaurant / dashboard
       |
       +-- reusable browser SDK: page views + custom events
       |
       +-- HTTP JSON requests (/api/*)
                    |
              Laravel API
              |           |
        validation      business logic
              |           |
              +----- SQLite database
                    events table
                    orders table
                         |
              Dashboard summary queries
```

Frontend

- src/main.jsx React UI, menu loading, order submission aur analytics dashboard render karta hai.
- src/analytics.js reusable browser tracking client deta hai.
- src/api.js API base URL ko ek jagah se banata hai. `VITE_API_BASE_URL` khaali ho to relative `/api/...` URL use hota hai.
- vite.config.js local development mein `/api` ko Laravel port `8000` par proxy karta hai.

Laravel backend

- backend/routes/api.php API routes define karta hai.
- `backend/app/Http/Controllers/Api/` ke controllers input validate karke API responses banate hain.
- backend/app/Services/Analytics/ServerEventTracker.php backend code se trusted analytics event likhne wala reusable service hai.
- `backend/database/migrations/` tables aur indexes ka schema define karta hai.
- backend/config/analytics.php server API key configuration deta hai.
- backend/config/cors.php alag origin se aane wali API requests ke allowed origins configure karta hai.

4. Event data model

Har event mein yeh fields hoti hain:

Field	Maqsad
`event_id`	Unique ID; duplicate submission ko pehchanne ke liye
`event_name`	Event type, jaise `page_view`, `menu_item_selected`, `order_created`
`visitor_id`	Browser ke anonymous visitor ko identify karta hai
`session_id`	Current browser tab/session ko identify karta hai
`source`	Backend assign karta hai: `browser` ya `server`
`page_path`	Optional relative path, jaise `/` ya `/dashboard`
`properties`	Event-specific JSON object, jaise order ID ya selected menu item
`occurred_at`	Event kab hua, UTC timestamp
`received_at`	Backend ne event kab receive kiya, UTC timestamp

Browser SDK visitor ID ko `localStorage` aur session ID ko `sessionStorage` mein rakhta hai. Dono UUID-based anonymous IDs hain; ye login/authentication IDs nahi hain. Browser storage clear hone ya doosre browser/device use karne par nayi identity ban sakti hai.

Example browser event:

```
{
  "events": [
    {
      "event_id": "e4128043-53d2-41b2-b688-a2a21fae0d33",
      "event_name": "page_view",
      "visitor_id": "visitor_123",
      "session_id": "session_456",
      "page_path": "/dashboard",
      "properties": {},
      "occurred_at": "2026-10-14T09:30:00.000Z"
    }
  ]
}
```

Browser se `source` accept nahi hota; Laravel request ke trust context ke hisaab se khud source set karta hai.

5. Browser tracking ka end-to-end flow

1. Restaurant ya dashboard component `usePageView()` ke zariye SDK ko page path deta hai.
2. SDK visitor/session IDs leta hai ya browser storage mein pehli baar UUID bana kar rakhta hai.

3. SDK unique `event_id` aur UTC `occurred_at` ke saath JSON event banata hai.
4. Browser `POST /api/events` call karta hai. Vite development proxy request Laravel tak bhejta hai.
5. Laravel batch, event name, IDs, page path, properties aur timestamp validate karta hai.
6. Laravel `source=browser` lagata hai. Browser `order_created` jaise trusted event ko submit nahi kar sakta.
7. Event `events` table mein insert hota hai. Existing `event_id` primary key hone se repeat event duplicate hota hai, doosri row nahi.
8. Dashboard Laravel se date-range summary mangta hai aur response ko display karta hai.

SDK `track()`, `pageView()` aur `event()` methods expose karta hai. Restaurant demo menu select hone par `menu_item_selected` custom event bhejta hai. Abhi SDK har tracked event ko alag request mein bhejta hai; ingestion endpoint ek request mein 1–50 events accept karta hai.

6. Server tracking aur order ka flow

1. React `POST /api/orders` ko item ID, quantity aur anonymous visitor/session IDs bhejta hai.
2. Laravel input validate karta hai. Client ka bheja hua price use nahi hota; price server ke menu data se liya jaata hai.
3. Laravel order ID banata hai aur `total_cents = server_price × quantity` calculate karta hai.
4. Database transaction shuru hota hai.
5. Order `orders` table mein insert hota hai.
6. `ServerEventTracker` `order_created` event ko `source=server` ke saath `events` table mein likhta hai.
7. Dono writes successful hon to transaction commit hota hai. Koi write fail ho to transaction rollback dono writes ko undo karta hai.
8. API `201 Created` ke saath order aur `analytics: "tracked"` return karti hai.

`ServerEventTracker` Laravel container se inject hota hai aur doosre backend business workflows bhi use kar sakte hain. Is current project mein yeh internal service hai: event ko API ko HTTP request bhejne ke bajaye directly same database mein likhta hai. `/api/events` endpoint external/trusted server clients ke liye `X-Analytics-Key` bhi support karta hai.

7. Database kahan hai aur tables kya hain?

Default local SQLite database file:

```
backend/database/database.sqlite
```

SQLite yahan server ko alag se install/run kiye bina single-process demo ko persistent database deta hai. Orders aur analytics events **ek hi database file** mein, lekin **alag tables** mein rehte hain.

`events`

- `event_id` primary key aur deduplication key hai.

- `event_name`, `visitor_id`, `session_id`, `source`, `page_path` aur `properties` event ka content rakhte hain.
- `occurred_at` aur `received_at` event time aur ingestion time alag rakhte hain.
- `occurred_at`, `(event_name, occurred_at)` aur `(page_path, occurred_at)` par indexes common reporting filters ko support karte hain.

`orders`

- `order_id` primary key hai.
- Item, quantity, server-calculated `total_cents`, visitor/session IDs aur `created_at` store hote hain.
- Money integer cents mein store hota hai taaki floating-point rounding se bacha ja sake.

Laravel framework migrations extra framework tables bhi bana sakti hain—jaise `migrations`, `users`, `cache` aur `jobs`. Assignment ka business data `events` aur `orders` tables mein hai. Schema analytics migration mein defined hai.

8. API endpoints aur unka kaam

Base path `/api` hai:

Method / path	Request	Kaam / response
`POST /api/events`	`{ "events": [...] }`	Browser ya API-key-authenticated server events validate, deduplicate aur save karta hai; `202` par `accepted` aur `duplicates` deta hai
`GET /api/analytics/summary?from=YYYY-MM-DD&to=YYYY-MM-DD`	Optional UTC date range	Totals, zero-filled daily counts, top 10 pages aur top 10 event types return karta hai
`GET /api/menu`	Koi body nahi	Menu IDs, names aur server-owned prices return karta hai
`POST /api/orders`	`item_id`, `quantity`, `visitor_id`, `session_id`	Server price se order create karke trusted `order_created` event save karta hai

Invalid input Laravel ke JSON validation error response ke saath reject hota hai (aam taur par `422`). Galat `X-Analytics-Key` `401` deta hai. Browser request mein header na ho to event browser source ke saath save hota hai; valid key wala request server source ke saath save hota hai.

9. Dashboard metrics ka matlab

- **Page Views:** selected UTC date range mein `page_view` events ka count.
- **Unique Visitors:** selected range mein kam-se-kam ek page view karne wale distinct anonymous `visitor_id` count.
- **All Events:** selected range ke sabhi event types ka total, page views samet.

- **Event Activity:** har din ke sabhi events aur page views; selected range ke har din ke liye zero-event day bhi dikhaya jaata hai.
- **Popular Pages:** page view count se ranked top 10 paths aur har path ke distinct visitors.
- **Event Mix:** event type ke hisaab se top 10 event counts.

`from` aur `to` inclusive UTC calendar dates hain. Maximum date range 90 din hai; start date end date ke baad nahi ho sakti.

Isliye ek page visit se Page Views `+1` aur All Events `+1` dono hote hain. All Events mein Page Views pehle se included hain; ye alag-alag categories nahi hain.

10. Validation, reliability aur security decisions

- Event batch 1–50 events tak limited hai; `/api/events` par 120 requests per IP per minute ki Laravel throttle middleware hai.
- `event_name`, visitor/session/event IDs, page path aur event time validate hote hain.
- Event time 30 din se zyada purana ya 5 minute se zyada future ka nahi ho sakta.
- Event properties ka serialized JSON size 4 KB tak limited hai.
- Browser se `order_created` event reject hota hai. Server source client payload se nahi, configured API key ke context se set hota hai.
- Production environment mein default `local-development-key` allowed nahi; production ke liye `ANALYTICS_SERVER_KEY` set karna zaroori hai.
- Duplicate `event_id` insert ko database primary key rokti hai. Batch ko pehle validate kiya jaata hai aur phir insert transaction mein hota hai.
- Order aur order event ek transaction mein commit/rollback hote hain.
- Browser IDs anonymous pseudonymous IDs hain, authenticated identity nahi. Public browser ingestion fabricated events ko poori tarah rok nahi sakta.

11. Local setup aur run karna

Requirements: PHP 8.2+, Composer, Node.js 20+, npm. PHP mein SQLite/PDO SQLite extension enabled honi chahiye.

Project root `M:\kitchenspurs` se PowerShell commands:

```
npm.cmd install
Set-Location backend
composer.bat install
Copy-Item .env.example .env
php artisan key:generate
php artisan migrate
Set-Location ..
npm.cmd run dev
```

`npm` ke bajaye `npm.cmd` PowerShell execution-policy ki `npm.ps1` error se bachne ka seedha tareeqa hai. `npm.cmd run dev` Laravel API ko port `8000` par aur Vite frontend ko aam taur par port `5173` par start karta hai. Agar port busy ho to Vite agla available port, jaise `5174`, dikhata hai; terminal mein dikhaya gaya URL browser mein kholen. Dono servers chalne tak terminal khula rakhen.

Production-style local check:

```
npm.cmd run build
npm.cmd run preview
```

Preview frontend ke liye doosre terminal mein Laravel start karein:

```
npm.cmd start
```

Trusted server-event API use karte waqt `backend/.env` mein `ANALYTICS_SERVER_KEY` ko apni secret value par set karein. Secret ko React/browser bundle mein kabhi na daalein. Production ya cross-origin frontend deployment ke liye `CORS_ALLOWED_ORIGINS` aur build time par `VITE_API_BASE_URL` configure karein.

12. Tests aur verification

```
npm.cmd test
```

Yeh `backend` mein Laravel/PHPUnit tests chalaata hai. Feature tests event validation aur duplicate handling, atomic batch validation, summary aggregation/zero-fill, date range validation, server-priced order, trusted server event aur menu response verify karte hain. React production bundle ke liye:

```
npm.cmd run build
```

PHP style check ke liye Laravel Pint available hai:

```
Set-Location backend
vendor\bin\pint.bat --test app\Http\Controllers\Api app\Services\Analytics routes\api.php
database\migrations tests\Feature
```

13. Assignment ke liye key design decisions

1. **Ek SQLite database, alag domain tables:** order aur event write ek transaction mein possible hai; demo ko separate DB service nahi chahiye.
2. **Anonymous identity:** authentication ki requirement nahi thi; first-party visitor ID aur per-tab session ID analytics ke liye simple pseudonymous identity dete hain.
3. **Server-owned prices:** order ka total browser se trust nahi kiya jaata.

4. **Idempotency:** stable event ID primary key duplicate delivery ko second event row banne se rokti hai.
5. **Client event aur business event mein farq:** page views aur interactions browser SDK se aate hain; confirmed order event backend se trusted source ke saath aata hai.
6. **UTC date handling:** event timestamps UTC mein save/query hote hain, aur dashboard date range inclusive hoti hai.
7. **Framework-first API:** Laravel validation, routing, database transactions, migrations aur testing conventions use kiye gaye hain.

14. Scope ke bahar / production mein kya badalna hoga?

- SQLite ek local, single-process demo ke liye hai; high write volume ya multiple app instances ke liye managed relational DB aur shared operational infrastructure evaluate karna hoga.
- Server tracker abhi synchronous database write karta hai; high-volume system mein durable queue/outbox aur retries useful honge.
- Rate limiting Laravel configuration par depend karti hai; multi-instance production mein shared cache-backed limiter configure karna chahiye.
- Consent management, authenticated user identity, retention/deletion policy, bot filtering, abuse monitoring, event export aur dashboard pagination is assignment scope se bahar hain.
- Browser identity clear ho sakti hai ya browsers/devices ke beech link nahi hoti; metrics anonymous visitor estimates hain.
- Browser ingestion public hai; API key ko browser mein rakhna safe nahi hoga. Server-only event claims trusted server credentials ke peeche rahte hain.

Yeh demo learning/assignment ke liye thoughtful baseline hai; production readiness ka claim nahi karta.

15. Interview mein 60–90 second explanation

> “I built a full-stack analytics demo using React, a reusable browser SDK, Laravel APIs, and SQLite. The browser SDK creates an anonymous visitor ID and a tab-scoped session ID, then sends page views and custom events to a validated ingestion endpoint. Laravel assigns the event source, validates timestamps and payload size, and deduplicates events using the event ID as the primary key.
>
> The restaurant order flow keeps pricing on the server. When an order is created, Laravel saves the order and records an `order_created` server event in the same database transaction, so they either both commit or both roll back. The dashboard queries the events table for page views, distinct visitors, popular pages, daily trends, and event mix over an inclusive UTC date range capped at 90 days.
>
> I chose SQLite because the assignment is a small local demo and it avoids requiring a separate database service. I documented the trade-offs: for higher write volume or multiple app instances I would evaluate a shared database, queue/outbox delivery, and shared rate limiting. I also added feature tests for validation, deduplication, order pricing, analytics tracking, and dashboard aggregation.”

16. Common interview follow-ups

“All Events aur Page Views dono ek visit par kyun badhte hain?”
Page view khud ek event type hai. Isliye `All Events` mein page views included hain. Ek single page load se dono metric +1 hote hain.

“Order event ko browser se kyun nahi bhejte?”
Browser request user control kar sakta hai. Confirmed order aur uski price trusted server-side business logic se aani chahiye; warna client fake order analytics bhej sakta hai.

“Duplicate events kaise rokte ho?”
Har event ka stable `event_id` hai aur `events.event_id` primary key hai. Same ID dobara aaye to doosri row insert nahi hoti; ingestion response accepted/duplicate counts deta hai.

“Page view exactly once ki guarantee hai?”
Nahi. Event ID duplicate delivery ko idempotent banata hai, magar reload/navigation ya SDK ko dobara trigger karna alag event IDs bana sakta hai. Yeh system page-load/page-view events record karta hai, exactly-once human visit nahi.

“Unique visitor actual person count hai?”
Nahi. Yeh selected period mein page view karne wale distinct anonymous browser IDs ka count hai. Storage clear karne ya doosre browser/device par aane se identity badal sakti hai.

“Event save failure par order ka kya hota hai?”
Current demo mein order aur event ek database transaction mein hain. Dono successful hon to commit; event write fail ho to order bhi rollback. Agar business ko order success analytics se zyada important ho, production design mein outbox/queue ya independent failure policy chuni ja sakti hai.

“Alag database kyun nahi?”
Assignment separate database nahi maangta. Same SQLite database mein `orders` aur `events` alag tables hain, jisse demo simple aur order-plus-event transaction atomic rehta hai. Higher scale par database choice phir se evaluate karunga.

17. Code map

File	Kya samjhata / implement karta hai
src/analytics.js	Browser tracking SDK
src/api.js	Frontend API base URL helper
src/main.jsx	Restaurant/order UI aur analytics dashboard
vite.config.js	Local frontend/API proxy
backend/routes/api.php	Laravel API route list
EventController.php	Event validation, source assignment, deduplication aur ingestion

File	Kya samjhata / implement karta hai
AnalyticsSummaryController.php	Date-range analytics queries
OrderController.php	Server-priced order aur transaction
ServerEventTracker.php	Reusable trusted server event writer
MenuController.php	Server-owned menu data
analytics migration	`events` aur `orders` table schemas/indexes
AnalyticsApiTest.php	Main API behavior ke feature tests